

Chapter 15 Abstract Classes and Interfaces

Part 1 (Abstract Classes and methods)



Objectives

To design and use abstract classes (§15.2).

To design and use **abstract methods**(§15.2).

To generalize numeric wrapper classes using the abstract **Number** class (§15.3).



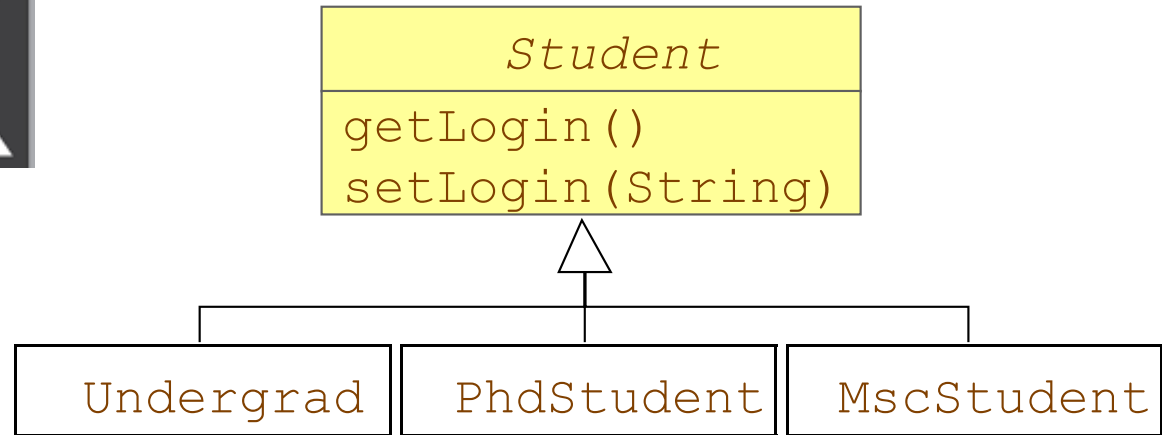


Do you think it is logical to create an object of class animal?

Do you think it is logical to create an object of classes Cat or Dog?

So why do we need the Animal Class?



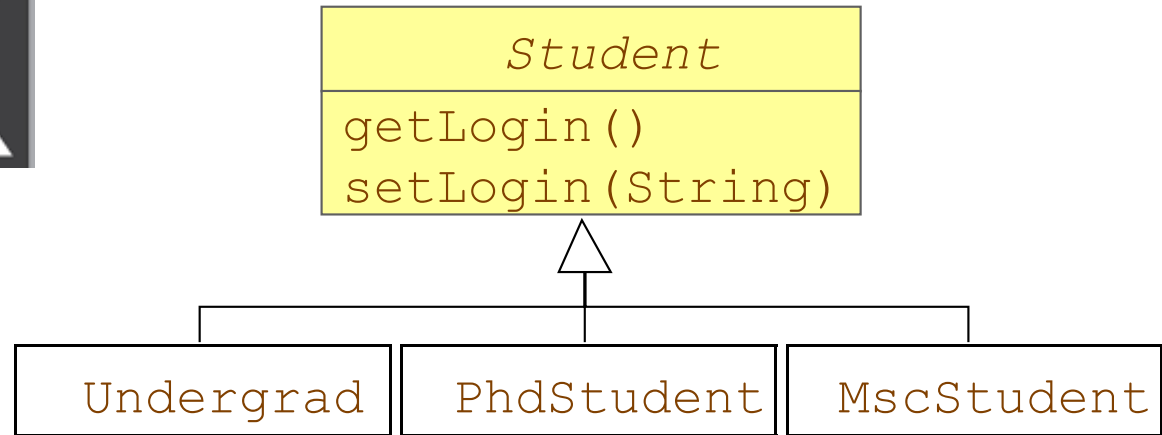


- F The problem:
- Students are either undergraduate, MSc or PhD.
 - We want to guarantee that nobody creates a Student object. The application always creates a specific kind of Student.

How can we apply the above requirements?

Declare Student as an abstract class.

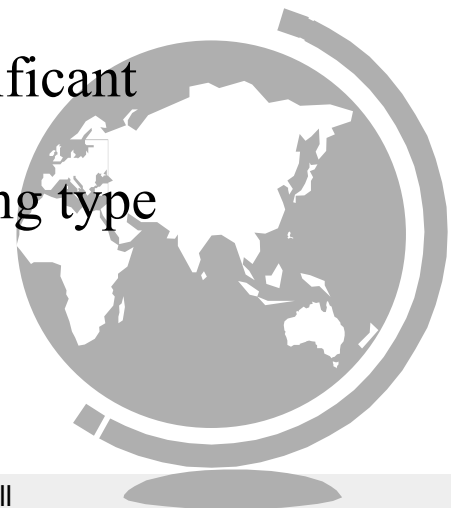




F

The problem:

- Students are either undergraduate, PhD or MsC.
- We want to guarantee that nobody creates a *Student* object. The application always creates a specific kind of *Student*.
- **Why do we have the *Student* class in the first place?**
- A common implementation of common aspects of all students (e.g. *setLogin()* and *getLogin()*).
- A place holder in my hierarchy that corresponds to a significant concept in the problem domain.
- To handle all students independently of their subclass using type *Student* and polymorphism.



Abstract classes

- F In inheritance hierarchy, super classes become more general, and subclasses become more specific.
- F You must declare the class with the keyword `abstract`:
`abstract class MyClass {...}`
- F An abstract class **cannot** be instantiated using the **new** operator.

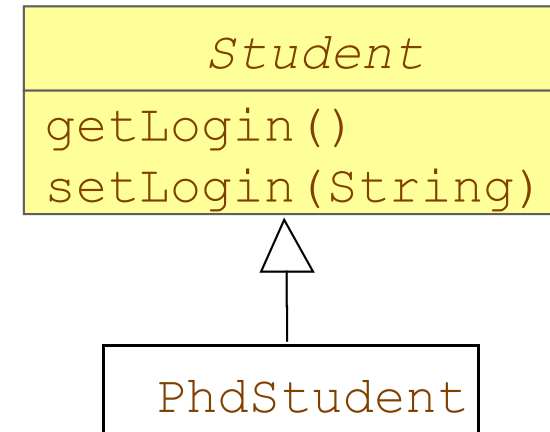


Abstract Classes in Java

```
public abstract class Student {  
    protected String login, department, name;  
  
    public Student() {  
        login = ""; department = ""; name = "";  
    }  
  
    public void setLogin(String login) {  
        this.login = new String(login);  
    }  
  
    public String getLogin() {  
        return new String(login);  
    }  
}
```

PhdStudent is said
to be a **concrete class**

```
public class PhdStudent extends Student{  
    private String supervisor;  
  
    public void setSupervisor(String login) {
```



Abstract Classes in Java

```
public abstract class Student {  
    protected String login, department, name;  
  
    public Student() {  
        login = ""; department = ""; name = "";  
    }  
  
    public void setLogin(String login) {  
        this.login = new String(login);  
    }  
  
    public String getLogin() {  
        return new String(login);  
    }  
}
```



BUT you can still define its constructors, which are invoked in the constructors of its subclasses.

Abstract Classes

For instance, the constructors of Shape are invoked in the Circle class and the Rectangle class.

Can you initiate a concrete object from Shape? Why?



abstract class as type

Can I define an array of abstract class, such as animal or shape?

YES

An abstract class can be used as a data type.

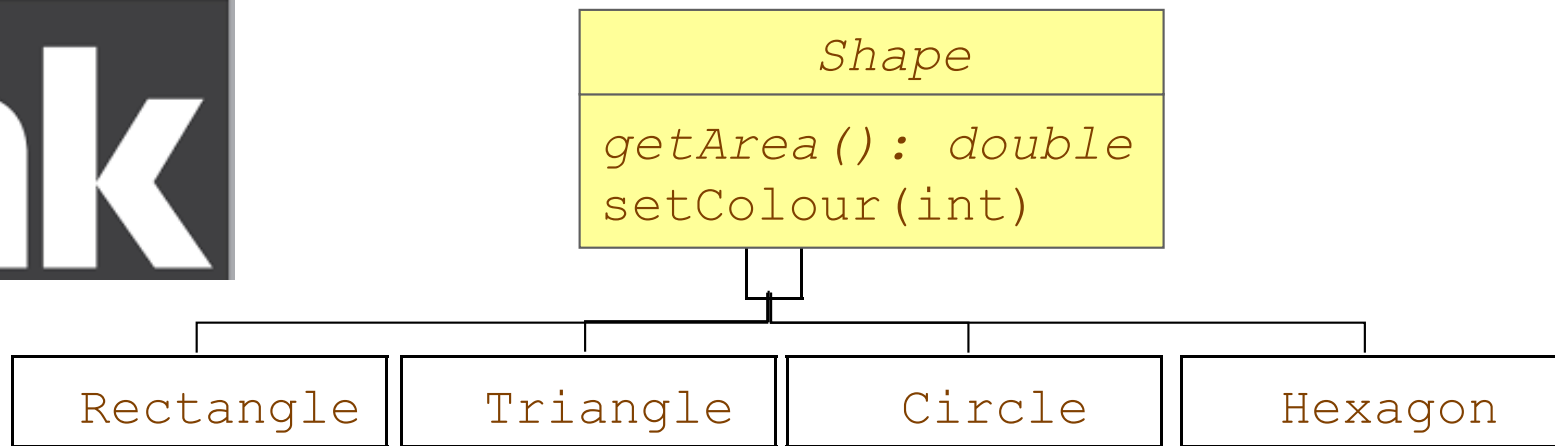
The following statement creates an array whose elements are of Shape type.

Shape[] geo = new Shape[10]; ✓

geo[0] = new Circle(); ✓

geo[1] = new Shape(); ✗





We cannot create *Shape* objects, because we cannot provide a reasonable implementation of `getArea()`; How do we calculate the area of an arbitrary shape?

- So we declare the *Shape* to be an **abstract** class.
- Furthermore, we declare `getArea()` as an **abstract method** **because** it has no implementation.

Why have the *Shape* class in the first place? And why we have to address the `getArea` in the *Shape* class?

we want to force all shapes to provide an implementation for `getArea()`;



Abstract methods

F You can *declare* an object without *defining* it:

```
Person p;
```

F Similarly, you can declare a *method* without defining it:

```
public abstract void draw(int size);
```

— Notice that the body of the method is missing

F A method that has been declared but not defined is an **abstract method**.



Abstract Methods in Java

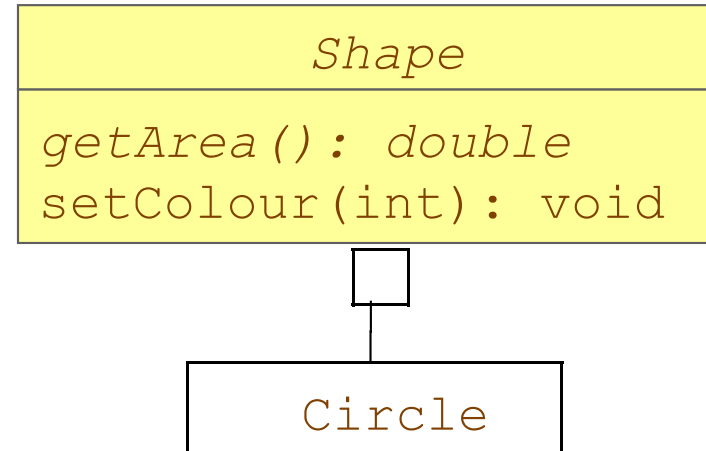
```
public abstract class Shape {
    final static int BLACK = 0;
    private int colour;

    public Shape() {
        colour = BLACK;
    }

    public void setColour(int c) {
        this.colour = c;
    }

    public abstract double
    getArea();
}
```

Abstract methods have no body



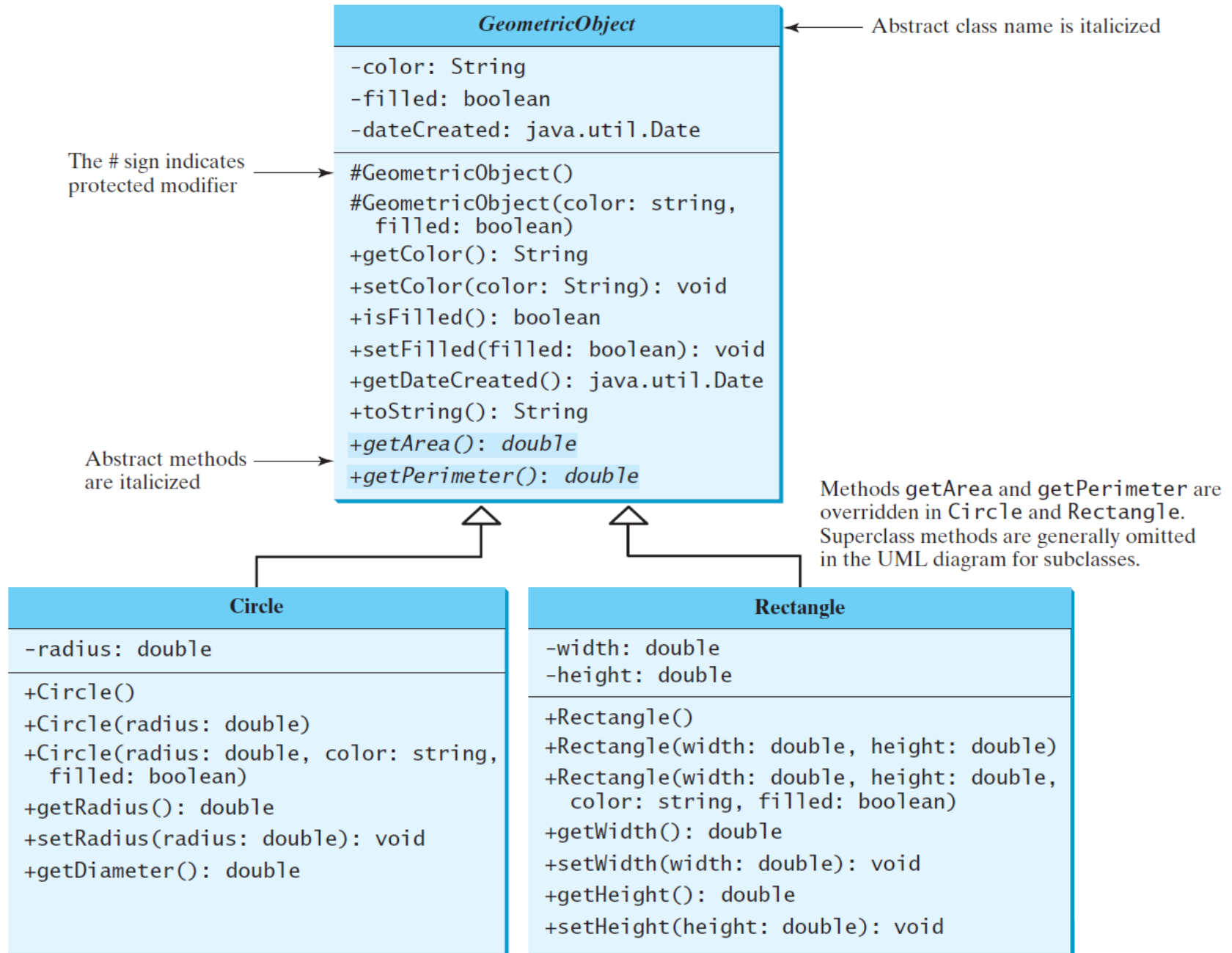
```
public class Circle extends Shape {
    final static double PI = 3.1419;
    private int radius;

    public Circle(int r) {
        radius = r;
    }

    public double getArea() {
        return (radius^2)*PI;
    }
}
```

If Circle did not implement getArea() then it would have to be declared abstract too!

Abstract Classes and Abstract Methods



abstract method in abstract class

- F If the subclass **defines** all the inherited **abstract methods**, it is “**complete**” and can be instantiated.
- F If the subclass does not define **all** the inherited abstract methods, it too **must be abstract**.
- F A class that contains **abstract methods** must be **abstract**.
- F In a **non-abstract** subclass extended from an abstract class, **all** the abstract methods **must be implemented**, even if they are not used in the subclass.



abstract method in abstract class

- F It is possible to define an abstract class that contains **no** abstract methods. In this case, you cannot create instances of the class using the new operator. This class is used as a base class for defining a new subclass.
- F A subclass can be **abstract** even if its superclass is **concrete**. Example?

The Object class is concrete, but its subclasses, such as Shape may be abstract.





```
class Shape { ... }  
class Star extends Shape {  
    void draw() { ... }  
    ...  
}  
class Circle extends Shape {  
    void draw() { ... }  
    ...  
}
```

Are the following codes correct?

Shape someShape = new Star(); This is legal, because a Star *is* a Shape

someShape.draw(); This is a syntax error, because *some Shape* might not have a `draw()` method
Remember: *A class knows its superclass, but not its subclasses*

Solutions?





```
abstract class Shape {  
    public abstract void draw();  
}  
class Star extends Shape {  
    void draw() { ... }  
    ...  
}  
class Crescent extends Shape {  
    void draw() { ... }  
}
```

Are the following codes correct?

Shape someShape = new Star(); This is legal, because a Star *is* a Shape
Shape someShape = new Shape(); *is no longer* legal

someShape.draw(); This is legal, because every actual instance
 must have a draw() method.





GeometricObject s = new GeometricObject(); **X**

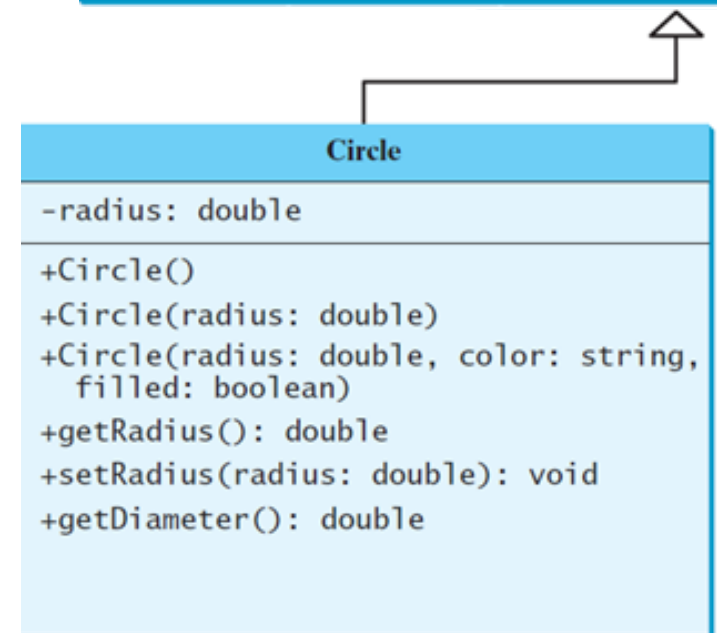
GeometricObject s = new Circle(4); ✓

double area = s.getArea(); ✓

Circle c = new Circle(3); ✓

c.setColour(GREEN); ✓

area = c.getArea(); ✓



concrete method overridden to be abstract

```
Class Object{  
    public String toString(){...}  
}  
Class Animal{  
    public abstract String toString();  
}
```

What happened?

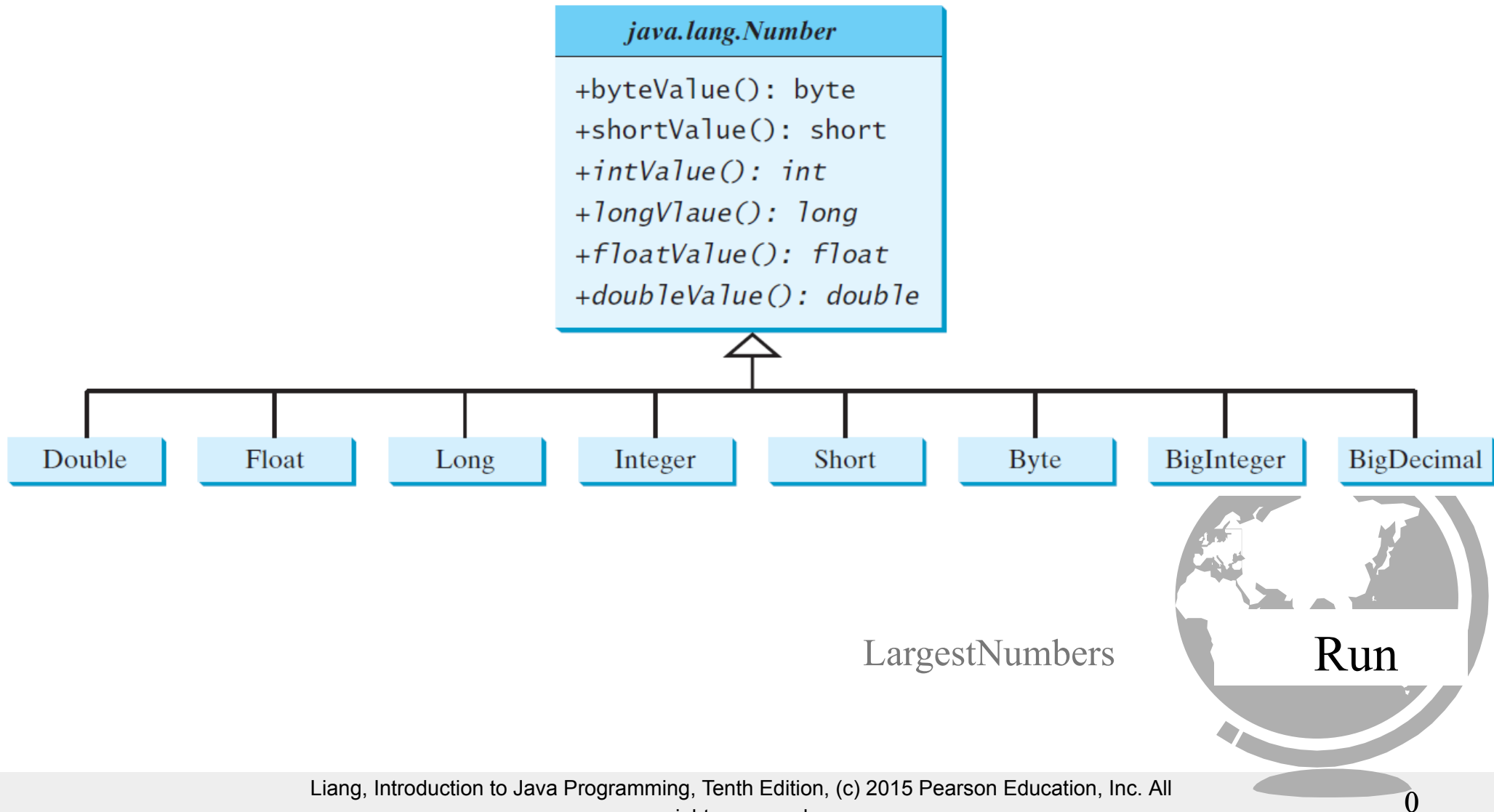
A subclass can override a method from its superclass to define it abstract.

Why? For what reason?

This is rare, but useful when the implementation of the method in the superclass becomes invalid in the subclass. In this case, the subclass must be defined abstract.



Case Study: the Abstract Number Class



```

public class LargestNumbers {
    public static void main(String[] args) {

        ArrayList<Number> list = new ArrayList<>();
        list.add(45);
        list.add(3445.53);
        list.add(33.9);
        list.add(46);

        System.out.println("The largest number is " + getLargestNumber(list));
    }

    public static Number getLargestNumber(ArrayList<Number> list) {
if (list == null || list.size() == 0)
        return null;

        Number number = list.get(0);
for (int i = 1; i < list.size(); i++)
        if (number.doubleValue() < list.get(i).doubleValue())
            number = list.get(i);
return number;
    }
}

```



